

INVESTIGACIÓN SPRING SECURITY

Login completo con Spring Boot + Spring Security + Thymeleaf

PROYECTO MICS — SISTEMA DE CONFIRMACIÓN DE INFORMACIÓN MÉDICA

Presentado por:

Hermes David Capacho Hernandez

Joseph David Vasquez Quintero

Presentado a:

Ing. Carlos Adolfo Beltran Castro

Unidades Tecnológicas de Santander

Tecnología en Desarrollo de Sistemas Informáticos

Programación en Java — Grupo B193

Bucaramanga, 2026

1. ¿Qué es Spring Security?

Spring Security es un Framework de seguridad para aplicaciones Java basadas en Spring. Proporciona autenticación (verificar quién eres) y autorización (verificar qué puedes hacer) de forma integrada con Spring Boot. Es el estándar de la industria para proteger aplicaciones web en el ecosistema Java.

En el proyecto MICS, Spring Security se encarga de proteger todas las rutas del sistema, de modo que solo los usuarios con credenciales válidas (médicos y administradores registrados) puedan acceder a la información médica de los pacientes.

2. Tecnologías Utilizadas

Tecnología	Versión	Función en el proyecto
Spring Boot	3.5.x	Framework base para crear la aplicación web.
Spring Security	6.x	Manejo de autenticación, autorización y protección de rutas.
Thymeleaf	3.x	Motor de plantillas para renderizar las vistas HTML del Login y el home.
Spring Data JPA	3.x	Conexión y manejo de la base de datos MySQL.
MySQL	8.x	Base de datos donde se almacenan los usuarios y datos del sistema MICS.
BCryptPasswordEncoder	-	Algoritmo de cifrado para las contraseñas de los usuarios.

3. Arquitectura de la Solución

La solución sigue el patrón MVC (Modelo - Vista - Controlador) propio de Spring Boot, con una capa adicional de seguridad gestionada por Spring Security. El flujo completo es el siguiente:

1. El usuario accede a cualquier URL del sistema (por ejemplo /home).
2. Spring Security intercepta la petición antes de que llegue al controlador.
3. Si el usuario no ha iniciado sesión, lo redirige automáticamente a /login.
4. El usuario ingresa su nombre de usuario y contraseña en el formulario Thymeleaf.
5. Spring Security valida las credenciales usando el UserDetailsService y BCrypt.
6. Si las credenciales son correctas, crea una sesión segura y redirige a /home.
7. Si son incorrectas, redirige a /login?error=true y muestra el mensaje de error.
8. Al cerrar sesión, Spring Security destruye la sesión y redirige a /login?logout=true.

4. Estructura del Proyecto

```
mics-login/
├── src/main/java/com/mics/login/
│   ├── MicsLoginApplication.java (clase principal)
│   ├── SecurityConfig.java      (configuración de Spring Security)
│   ├── AuthController.java      (controlador del login)
│   └── HomeController.java      (controlador del home)
├── src/main/resources/
│   ├── templates/
│   │   ├── login.html          (vista del formulario de login)
│   │   └── home.html           (vista del panel principal)
│   └── application.properties  (configuración de BD y servidor)
└── pom.xml                    (dependencias Maven)
```

5. Explicación de Cada Componente

5.1 SecurityConfig.java

Es el archivo más importante. Aquí se define toda la configuración de seguridad del sistema. Tiene tres métodos principales:

- `passwordEncoder()`: define el algoritmo BCrypt para cifrar las contraseñas. BCrypt genera un hash diferente cada vez, lo que hace imposible revertir la contraseña original.
- `userDetailsService()`: define los usuarios del sistema en memoria. En esta etapa del proyecto se usan usuarios en memoria (`InMemoryUserDetailsManager`). En fases futuras se conectará a la tabla CUENTA de la base de datos MySQL.
- `securityFilterChain()`: define las reglas de acceso. Indica qué rutas son públicas (`/css`, `/js`, `/images`), cuál es la página de login personalizada (`/login`), a dónde redirigir tras login exitoso (`/home`) y qué pasa al cerrar sesión.

```
@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
    http
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/css/**", "/js/**").permitAll()
            .anyRequest().authenticated()
        )
        .formLogin(form -> form
            .loginPage("/login")
            .defaultSuccessUrl("/home", true)
            .failureUrl("/login?error=true")
            .permitAll()
        );
    return http.build();
}
```

5.2 AuthController.java

Controlador encargado de manejar las peticiones GET hacia /login. Recibe dos parámetros opcionales de la URL:

- error: cuando Spring Security detecta credenciales incorrectas, redirige a /login?error=true. El controlador captura este parámetro y agrega el mensaje de error al modelo para que Thymeleaf lo muestre.
- logout: cuando el usuario cierra sesión, Spring Security redirige a /login?logout=true. El controlador agrega el mensaje de confirmación.

El método retorna el nombre de la plantilla 'login', lo que indica a Thymeleaf que debe renderizar el archivo login.html.

5.3 HomeController.java

Controlador que maneja la ruta /home. Recibe el objeto Authentication de Spring Security, que contiene la información del usuario autenticado. Extrae el nombre del usuario con authentication.getName() y lo envía al modelo para que Thymeleaf lo muestre en el mensaje de bienvenida.

5.4 login.html (Thymeleaf)

Plantilla HTML del formulario de login. Los elementos clave de Thymeleaf son:

- th:action="@{/login}": genera la URL correcta del formulario. Spring Security intercepta automáticamente el POST a /login.
- th:if="\${errorMsg}": muestra el div de error solo si existe el atributo errorMsg en el modelo.
- th:text="\${errorMsg}": inserta dinámicamente el texto del mensaje de error.
- El campo name="username" y name="password" son requeridos por Spring Security para identificar las credenciales.

```
<form th:action="@{/login}" method="post">
  <input type="text" name="username" />
  <input type="password" name="password" />
```

```
<button type="submit">Ingresar</button>  
</form>
```

5.5 home.html (Thymeleaf)

Plantilla HTML del panel principal. Solo es accesible si el usuario está autenticado. Muestra el nombre del usuario con `th:text="${usuario}"` y tiene un formulario de logout que hace POST a `/logout`, ruta interceptada por Spring Security para destruir la sesión.

5.6 application.properties

Archivo de configuración central del proyecto. Define la conexión a la base de datos MySQL (`mics_db`), el dialecto de Hibernate para MySQL, el puerto del servidor (8080) y la política de actualización automática del esquema (`ddl-auto=update`).

6. Flujo de Autenticación Detallado

Paso	Componente	Descripción
1	Navegador	El usuario accede a <code>http://localhost:8080/home</code> .
2	Spring Security Filter	Intercepta la petición. Detecta que el usuario no tiene sesión activa.
3	Spring Security	Redirige automáticamente a <code>/login</code> .
4	AuthController	Maneja el <code>GET /login</code> y retorna la vista <code>login.html</code> .
5	Thymeleaf	Renderiza el formulario de login en el navegador.
6	Navegador	El usuario ingresa usuario y contraseña y hace clic en Ingresar.
7	Spring Security Filter	Intercepta el <code>POST /login</code> . Extrae <code>username</code> y <code>password</code> del formulario.
8	UserDetailsService	Busca el usuario en memoria (o BD) y carga sus datos.
9	BCryptPasswordEncoder	Compara la contraseña ingresada con el hash almacenado.
10	Spring Security	Si es correcta: crea sesión y redirige a <code>/home</code> . Si es incorrecta: redirige a <code>/login?error=true</code> .
11	HomeController	Maneja <code>GET /home</code> , obtiene el nombre del usuario autenticado y lo pasa a la vista.
12	Thymeleaf	Renderiza <code>home.html</code> con el nombre del usuario en el mensaje de bienvenida.

7. Credenciales de Prueba

Para probar el sistema se configuraron los siguientes usuarios en memoria, asociados a los administradores registrados en la base de datos mics_db:

Usuario	Contraseña	Rol
admin_hermes	Pass1234!	ADMIN
admin_joseph	Pass5678!	ADMIN

Las contraseñas se almacenan cifradas con BCrypt. Nunca se guarda la contraseña en texto plano.

8. Conclusión

La implementación del módulo de login con Spring Security para el proyecto MICS cumple con los requerimientos de seguridad definidos en el documento inicial del proyecto (RF-11: autenticación mediante login con roles). El sistema protege todas las rutas sensibles, cifra las contraseñas con BCrypt y maneja correctamente los mensajes de error y cierre de sesión.

Como siguiente paso en el desarrollo del proyecto, se conectará el módulo de autenticación directamente con la tabla CUENTA de la base de datos MySQL, de modo que los administradores registrados puedan iniciar sesión con las credenciales almacenadas en mics_db, cumpliendo con la arquitectura definida en el MER y el diagrama de clases UML del proyecto.